It Takes Two to Tango: Accelerating Code Generation via Cache-Inspired Hierarchical Speculative Decoding

Anonymous Author(s)

Abstract

While Large Language Models (LLMs) have demonstrated exceptional performance in code generation, their autoregressive decoding paradigm typically results in low inference efficiency. Speculative decoding has emerged as a solution to speed up inference through its draft–verification mechanism, where multiple draft tokens are generated efficiently and then verified in parallel by LLMs. Current speculative decoding approaches for code generation are broadly categorized into two types: model-based approaches that **generate** drafts using a lightweight draft model, and retrieval-based approaches that **reuse** code snippets from external corpora. In this work, we first conduct empirical experiments to analyze the strengths and weaknesses of retrieval-based and model-based drafting. We find that retrieval-based approach is efficient but usually low-quality, while model-based one is high-quality but the drafting time increases with inference steps of the draft model. Inspired by the cache mechanism in computer architecture, we propose a novel framework **CacheCoder**. In our design, retrieval acts as a high-speed Level-1 (L1) cache for rapid draft acquisition, while the draft model functions as a more capable Level-2 (L2) cache to compensate for L1 misses, enabling us to capitalize on the speed of reuse when possible and rely on the precision of generation when necessary. To combine these two hierarchical processes, we propose two customized strategies, i.e., *Preliminary Prediction* and *Draft Expansion*, which utilize the draft model to pre-filter retrieved contents and expand them with tokens generated by the draft model to form drafts with higher length. Experimental results show that CacheCoder achieves up to a 2.64× speedup compared to autoregressive decoding, significantly outperforming state-of-the-art approaches.

CCS Concepts

• **Software and its engineering → Software maintenance tools**.

Keywords

Code generation, Speculative decoding, Large language models

ACM Reference Format:

Anonymous Author(s). 2018. It Takes Two to Tango: Accelerating Code Generation via Cache-Inspired Hierarchical Speculative Decoding. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

In the software development process, code generation is a technique used to assist the programming workflow. It enables the generation of relevant code snippets from natural language descriptions, providing developers with valuable references and significantly accelerating the development process. In recent years, large language models (LLMs) have advanced rapidly [4, 36], and Code LLMs, such as CodeLLama [32], Deepseek-Coder [8], and StarCoder [24], have achieved remarkable performance on code generation tasks [15, 17, 18, 40, 44].

Although many works have improved the effectiveness of LLM-based code generation, these approaches typically overlook the generation efficiency of LLMs during inference [33]. In practice, LLMs commonly adopt an autoregressive decoding paradigm, where at each decoding step the model generates only a single token conditioned on the current input. This decoding strategy fails to fully exploit the parallel computation capabilities of GPUs, resulting in low decoding efficiency [34]. In recent years, speculative decoding has been proposed to accelerate LLM inference without degrading generation quality [5, 21], using a draft–verification mechanism that generates multiple candidate tokens efficiently and verifies them in parallel with the target model (i.e., the model to be accelerated). Building on this idea, various approaches have been developed to speed up LLM-based code generation, and can be broadly categorized into two types, i.e., model-based and retrieval-based. On the one hand, model-based speculative decoding approaches typically employ an autoregressive model (known as the *draft model*) for drafting [25, 26, 46], which can be considered as a **generation** process. On the other hand, retrieval-based approaches typically introduce an external corpus and directly take the retrieved contents as the draft, which can be considered as a **reuse** process [10, 14]. For example, REST [10] retrieves code snippets from a code corpus that match the suffix of the current generation context and uses their subsequent segments as draft tokens for verification.

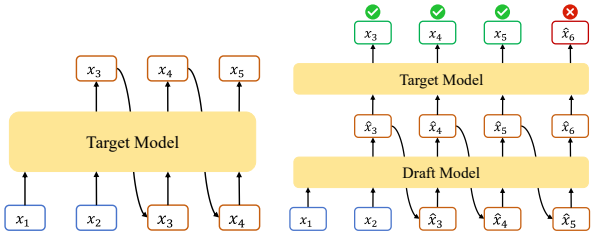
Recent studies [38] suggest that code generation and reuse are inherently complementary: while retrieval can provide ready-made solutions, generation is indispensable for creating new logic. Therefore, we hypothesize that reasonably fusing generation and reuse strategies during draft construction can lead to a more effective speculative decoding process. However, effectively fusing these two strategies in speculative decoding remains challenging, as it is difficult to predict a priori which strategy will perform better for a given context. Our core observation through our empirical analysis (will be detailed in Section 2.2) is a clear trade-off between efficiency and quality within these strategies. Reusing existing code is highly efficient as it bypasses heavy vector computations, yet it often suffers from high rejection rates due to misalignments with the target model [45]. Conversely, generating drafts with a smaller model ensures better alignment and higher draft quality but remains bottlenecked by the autoregressive process.

Inspired by the multi-level cache hierarchy in the classical computer architecture [11], we propose a novel speculative decoding framework that treats reuse and generation as hierarchical tiers. In our design, **the reuse strategy acts as a Level-1 (L1) cache**: it offers the fastest draft acquisition but possesses limited "hit rates" (i.e., the poor alignment with the target model). When an L1 "cache miss" occurs (i.e., the retrieved draft is deemed insufficient), the system falls back to **the generation strategy, acting as a Level-2 (L2) cache**, which is more computationally expensive but provides higher-quality drafts. The target model functions as the Comparator, performing the final verification. Furthermore, to avoid overwhelming the target model with verification requests, we devise two customized strategies, i.e., *Preliminary Prediction* and *Draft Expansion*, where the draft model serves as a pre-evaluator to identify reusable segments from the L1 cache. These segments are then seamlessly integrated with additional tokens generated by the draft model to ensure that each verification draft maintains a substantial sequence length, thereby reducing the invocation frequency of the target model. This approach leverages the strengths of both strategies: we capitalize on the speed of reuse when possible and rely on the precision of generation when necessary, effectively bypassing the aforementioned hard prediction problem.

To evaluate our approach, i.e., `CacheCoder`, we adopt CodeLlama-34B [32] and DeepSeek-Coder-33B [8] as the target models (a.k.a., the backbone models), and conduct systematic experiments on the MBCPP [1], MBGP [1], MBJSP [1], and MBPP [2] datasets. The results show that `CacheCoder` achieves up to 2.64× and 2.61× inference acceleration on CodeLlama-34B and DeepSeek-Coder-33B, respectively. Under the greedy decoding setting, it outperforms the current state-of-the-art approach Eagle-3 [26] by 19.3% in speed, while also achieving a 181% increase in acceptance length (i.e., the length of draft token accepted by the target model) and a 58% improvement in acceptance rate (i.e., the ratio between accepted tokens and draft tokens). Further ablation studies show the importance of combining reuse and generation processes. Finally, `CacheCoder` exhibits stronger generalization capability compared to Eagle-3 on the Multilingual HumanEval [1] dataset. Moreover, inserting the input and generated context as an extra cache component [45], the performance of `CacheCoder` can be further improved.

In summary, our main contributions are as follows:

- **Empirical Evidence**: Through empirical analysis, we observe that retrieval-based approaches can generate drafts efficiently but often suffer from relatively low draft quality, whereas generation-based approaches are able to produce higher-quality drafts at the cost of slower drafting speed. This complementary trade-off motivates us to combine retrieval and generation for draft construction.
- **Approach**: Inspired by the cache mechanism in computer architecture, we propose **`CacheCoder`**, which treats retrieval as an L1 cache and the generative draft model as an L2 cache. By reusing high-quality retrieved code snippets and supplementing them with the draft model, `CacheCoder` achieves a good balance between drafting speed and quality.
- **Evaluation**: We conduct experiments to systematically validate the effectiveness of our approach, achieving up to **2.64**× speedup compared to autoregressive decoding.



(a) Autoregressive decoding (b) Speculative decoding

Figure 1: Illustration of Autoregressive decoding and Speculative decoding.

2 Background and Motivation

2.1 Speculative Decoding

LLM inference typically employs autoregressive generation, where each decoding step generates one token, as shown in Figure 1a. This process is memory-bound and fails to fully leverage the computational power of GPUs [34], leading to low decoding efficiency.

Speculative decoding is a lossless method for accelerating LLM inference [5, 21]. It typically consists of two stages: a drafting stage and a verification stage. In the drafting stage, a highly efficient draft model is used to quickly generate multiple tokens. In the verification stage, the target model performs a parallel forward pass and accepts these tokens, thereby enabling the generation of multiple tokens in a single decoding step, as illustrated in Figure 1b.

Model-based approach. Specifically, let the target model be denoted as M_{target}, and the current input token sequence is denoted as $X_{1:t} = \{x_1, x_2, \ldots, x_t\}$. The draft model is denoted as M_{draft}. Considering greedy decoding, at each decoding step, M_{draft} autoregressively generates k tokens $\hat{X}_{t+1:t+k} = \{\hat{x}_{t+1}, \hat{x}_{t+2}, \ldots, \hat{x}_{t+k}\}$ based on $X_{1:t}$. In the verification stage, $\hat{X}_{t+1:t+k}$ is concatenated with $X_{1:t}$ and fed into the target model M_{target} to obtain $k+1$ tokens $X_{t+1:t+k+1}$ in parallel, where x_{t+i+1} is derived from $\hat{x}_{t+i}$. Formally, for x_{t+i}, it is obtained as follows:

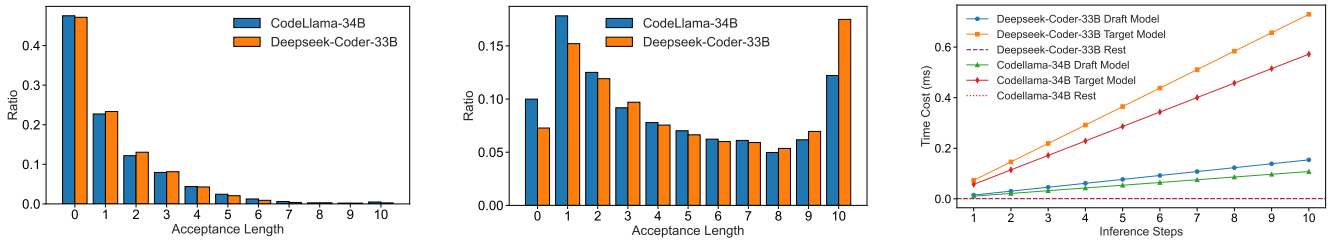
$$\begin{aligned} p_i &= M_{\text{target}}(x_{t+i} \mid X_{1:t}, \hat{X}_{t+1:t+i-1}), \\ x_{t+i} &= \text{argmax}\ p_i, \quad i = 1, 2, \ldots, k+1. \end{aligned} \tag{1}$$

If the drafted tokens $\hat{X}_{t+1:t+i}$ are identical to $X_{t+1:t+i}$, these i tokens are accepted as the output of speculative decoding. The verification terminates when either $i = k$ (all k draft tokens are accepted) or a mismatch $x_{t+i+1} \neq \hat{x}_{t+i+1}$ is encountered. Finally, x_{t+i+1} is additionally appended to the output sequence.

Retrieval-based approach. Typically, these approaches replace the autoregressive draft model with an efficient retrieval model. Specifically, such approaches usually construct a corpus from a general dataset [10] or the current input context [14, 45], and retrieve relevant content as the draft via string matching. Since these approaches can retrieve all draft content in a single step without requiring autoregressive generation by a small draft model, they usually achieve a highly efficient drafting speed.

2.2 Motivation

The basic idea of this work is to devise an approach that can effectively combine **reuse (i.e., retrieval-based draft)** and **generation**



(a) The ratio of acceptance length of draft code tokens retrieved by REST.

(b) The ratio of acceptance length of tokens generated by the draft model when REST fails.

(c) The time cost for the target model, the draft model, and REST.

Figure 2: Comparison between the draft model and Rest in terms of draft quality and time cost.

(i.e., model-based draft) strategies during the construction of the draft. To this end, we first perform an empirical study to better understand the strengths and weaknesses of these two types of strategies. Specifically, we conducted experiments on the MBXP [1] validation dataset [9] using CodeLlma-34B [32] and Deepseek-Coder-33B [8] as backbone models to evaluate the draft quality and draft time of the retrieval-based approach (i.e., REST [10]) and our constructed draft model (which will be detailed in Section 3.3).

First, to investigate the quality of retrieval-based and generation-based drafts, we compare their acceptance lengths verified by the target model. Figure 2a shows the ratio of different acceptance lengths for REST. We observe that the amount of reusable retrieved code is limited, where nearly 50% of the retrieved code is completely rejected by the target model. Figure 2b presents the distribution of acceptance lengths for the draft model when REST accepts zero tokens and indicates that the draft model is able to generate code with a longer acceptance length, thereby serving as a useful complement to REST. Next, to explore the efficiency between retrieval and generation, we compare their drafting time cost. As illustrated in Figure 2c, it can be observed that while the retrieval time of REST approaches zero for both CodeLlama-34B and Deepseek-Coder-33B, the cost of the draft model grows linearly with the number of inference steps. When the inference step becomes sufficiently large, the cost of the draft model can even exceed that of a single forward pass of the target model. These results suggest that the drafts generated by the draft model typically exhibit high quality; yet, as the draft model is usually autoregressive, the drafting time increases with the number of inference steps. In contrast, REST can obtain all drafts in one fast retrieval, but the quality of the retrieved code is generally lower.

With our observations, we propose to combine the strengths of the reuse and generation strategies. We draw inspiration from the design of caches in computer architecture, which accelerate CPU memory access [11]. In such systems, multiple levels of cache are placed between the CPU and main memory: the closer a cache is to the CPU, the faster its access speed, but the smaller its capacity and the lower its data hit rate. As shown in Figure 3, we treat reuse strategy as L1 cache which can fast retrieve drafts with relative low-quality, and view generation strategy as L2 cache which can generate more precise drafts but usually cost more draft time. We hypothesize that, by combining the efficiency of retrieval with the stronger capabilities of the draft model, we can achieve a balance between drafting quality and speed.

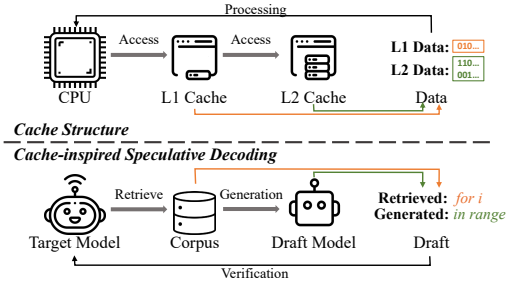


Figure 3: Illustration of hierarchical draft.

3 Approach

3.1 Overview

In this section, we propose **CacheCoder**, whose workflow is illustrated in Figure 4. Specifically, CacheCoder consists of three key components: (1) **Retrieval-Based Drafting**, which can quickly retrieve potentially reusable codes from the corpus, acting as the L1 cache; (2) **Self-Draft Model**, which is built from the intermediate layers of the target model and can generate higher-quality code drafts, acting as the L2 cache; (3) **Target Model**, which is the model we aim to accelerate and is responsible for verifying the drafts. Furthermore, we propose two strategies, i.e., *Preliminary Prediction* and *Draft Expansion*, to effectively coordinate the retrieval and generation processes. *Preliminary Prediction* first uses the draft model as an evaluator to select the high-quality tokens from the retrieved code, analogous to predict which content in the L1 cache is hit, and *Draft Expansion* further extend the selected code by the draft model, similar to fetching missing content from the L2 cache to compensate for L1 misses and thus maximizing the draft length.

3.2 Retrieval-Based Drafting

The pipeline of the retrieval-based drafting is illustrated in Figure 5. We first construct a retrieval corpus $D = \{(c_i, t_i)\}$ from The Stack [20], where c_i denotes the context and t_i denotes its subsequent code. We split each code snippet at each position to form context-target pairs (c_i, t_i). In the drafting stage, given the input

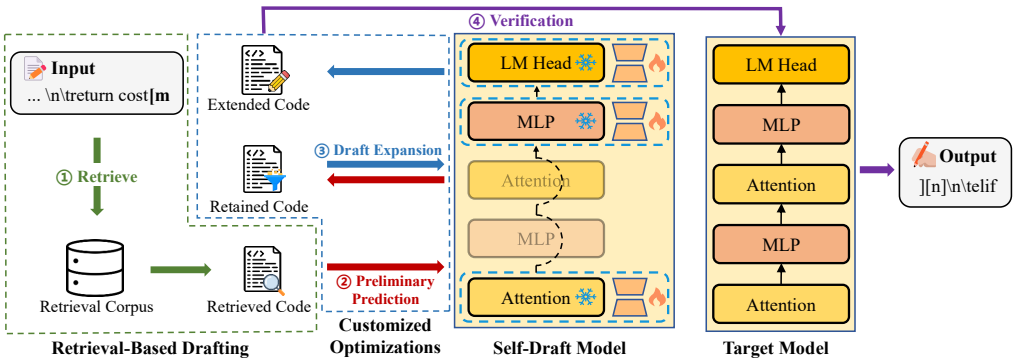


Figure 4: The overall workflow of CacheCoder

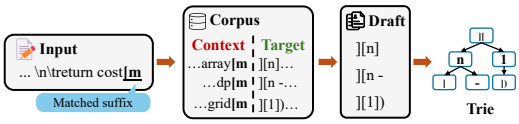


Figure 5: The pipeline of retrieval-based drafting

context $X_{1:t}$, we first identify the longest matched suffix $X_{t-i:t}$ and the corresponding c_i in D with the same suffix and uses its subsequent t_i as the retrieved draft, where $i \in [0, n_{max}]$ is the length of the matched suffix. Since retrieved candidates frequently exhibit substantial prefix overlap, we build a Trie (prefix tree) over these candidates to factor out shared prefixes. In the Trie, each root-to-node path corresponds to a prefix. For example, in the Figure 5, the path from the root "][" to the node "n" represents the prefixes of the sequences "][n]" and "][n-".

3.3 Distilled Self-Draft Model

We construct the draft model by retaining a subset of layers from the target model and performing a lightweight post-training. The details are as follows.

Retained-Layer Selection. Previous work [43] takes the overall decoding speed as the optimization objective, which balances the speed and quality of the drafting process, and obtains the optimum it via Bayesian optimization [19]. However, the optimal number of retained layers is typically about half of the total layers. Although this preserves the capability of the draft model to a large extent, it leads to high inference overhead for the draft model, which limits the upper bound of inference acceleration. To account for the efficiency of the draft model, our idea is to compress the scale of the draft model as much as possible, while leveraging a post-training to maintain the draft quality. Specifically, we incorporate a penalty term related to the draft model into the optimization objective. We define $z \in \{0,1\}^{2L}$ to indicate whether an attention layer or an MLP layer is retained, where 0 means the layer is kept, 1 means the layer is skipped, and L is the total number of model layers. Let $\mathrm{A}(z)$ denote the acceptance rate of the draft model, which is defined as the proportion of draft tokens that are accepted by the target model. Let $\mathrm{S}(z)$ denote the ratio of inference speed between the target model and the draft model, and $\mathrm{C}(z)$ represent the parameter compression ratio between the draft model and the target model. Our optimization objective is defined as:

$$z^* = \arg\max_z \Big(\mathrm{A}(z) - \alpha \cdot \max\big(0, \mathrm{S}(z) - \mathrm{S_t}\big)^2 - \beta \cdot \max\big(0, \mathrm{C}(z) - \mathrm{C_t}\big)^2 \Big) \tag{2}$$

Here, $\max(0, \mathrm{S}(z) - \mathrm{S_t})^2$ and $\max(0, \mathrm{C}(z) - \mathrm{C_t})^2$ acts as a regularization term, where $\mathrm{S_t}$ denotes the preset target ratio of inference speed between the target model and the draft model and $\mathrm{C_t}$ denotes the preset target parameter compression ratio of the draft model relative to the target model. α and β controls the penalty strength. $\mathrm{A}(z)$ encourages a high acceptance rate to maintain the draft model's capability, while the penalty term constrains excessive inference latency and parameter size of the draft model. In this way, Bayesian optimization can balance the acceptance rate and efficiency of the draft model.

Post-Training. After obtaining the optimal retained layers, we perform a lightweight post-training to better align the draft model with the target model. Specifically, we feed the prompts from the training set (refer to Section 4.2) into the target model to get their outputs as distillation data. We employ forward KL divergence as the loss function to distill the draft model, which has been demonstrated to be effective [46, 47]. To facilitate better alignment between the draft and target models, during training, the draft model will reuse the KV cache of the target model at each step when possible. To achieve this, we first randomly partition the output tokens of the distillation data into multiple blocks, where the tokens in the same block simulate those generated by the draft model in one draft step. Then, tokens within the same block use the KV cache of the draft model, while tokens across different blocks, which represent the corresponding context for the draft tokens, reuse the KV cache of the target model. Furthermore, we employ low-rank adaptation (LoRA) [13] to conduct model distillation, which only introduces a small number of extra parameters. During inference, LoRA is activated in the drafting stage and deactivated in the verification stage, thereby ensuring the output of target model remains unchanged.

3.4 Customized Optimizations

We have introduced retrieval-based and generation-based drafting, which serve as the "L1 cache" and "L2 cache", respectively.

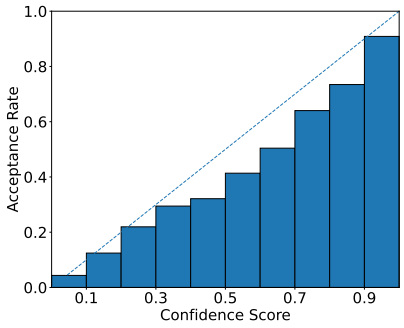


Figure 6: The relationship between the confidence score of the draft model and the acceptance rate of the target model.

As discussed in Section 2.2, we hypothesize that combining these two drafts can strike a balance between drafting quality and speed. However, unlike caches in computer architecture, where the cache controller coordinates all cache levels without CPU involvement, determining hits in the L1 and L2 caches requires invoking the target model for verification, which would significantly influence the efficiency. To alleviate the computational bottleneck imposed by excessive invocations of the target model, our approach incorporates two key customized optimizations, i.e., *Preliminary Prediction* and *Draft Expansion*. First, we leverage the draft model to predict L1-level hits, which acts as a preliminary filter to exclude low-quality retrieval results. Second, we deliberately extend the sequences sent to the target model for verification, aiming to maximize the efficiency of each validation cycle.

Draft Model for *Preliminary Prediction*. To predict which parts of the retrieved draft are hits, we use the draft model as an evaluator to replace the target model invocation. As the draft model is distilled from the target model, we argue that these two models exhibit strong alignment, which we will verify experimentally (see Section 6.2). Thus, tokens verified by the draft model are predicted as hits. Yet, to mitigate for the potential mis-prediction by the draft model, we decide to loosen the constraints. The key observation is that the confidence score of the draft model can also serve as an important indicator to evaluate the quality of the retrieved code. Given a sequence $\{x_1, \ldots, x_k\}$ generated by the draft model and the corresponding probabilities $\{p_1, \ldots, p_k\}$ output by the draft model, we define $c_i = \prod_{j=1}^{i} p_j$ as the confidence score for token x_i. Figure 6 shows the relationship between the confidence score and the acceptance rate of the target model (we calculate the averages of CodeLlma-34B and Deepseek-Code-33B), which is conducted on the validation set in our evaluation (details can be referred to Section 4.2). We observe a strong positive correlation between them, which means a higher c_i can imply a higher quality of x_i, and therefore we can also regard tokens with a confidence score greater than a threshold θ as the high-quality part and preserve them.

Overall, we treat two parts of the retrieved draft as hits and retain them for the next process: first, the tokens verified by the draft model; and second, the tokens whose confidence score from the draft model exceeds θ. The remaining parts in the retrieved draft are filtered out and regarded as misses.

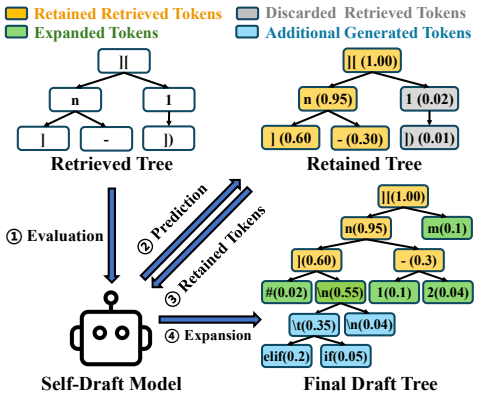


Figure 7: An example of *Preliminary Prediction* and *Draft Expansion*.

Draft Expansion. After the *Preliminary Prediction*, the volume of retained retrieval content may become significantly sparse. Validating such limited candidates directly against the target model would lead to a prohibitive increase in invocation frequency. To counteract this, we employ the draft model to expand the preserved tokens. This strategy is computationally efficient because the draft model has already performed an initial decoding pass during the preliminary prediction phase; thus, the incremental cost of draft expansion is marginal and does not significantly impact the overall draft-side latency. Specifically, leveraging the existing probability distributions, we can select the top-k candidates with the highest probabilities to expand each retained token without requiring additional inference. Then, we conditionally continue to expand each current leaf token t_i via the additional generation of the draft model if its confidence score satisfies $c_i \geq \theta$. This process terminates when either of the following conditions is met: (1) the length or number of the draft reaches the preset maximum value; (2) the confidence scores of all leaf tokens fall below θ, preventing generation with tokens that are unlikely to be accepted by the target model.

With these two strategies, we can achieve a good balance between draft quality and inference speed: when sufficient reusable code is available, the number of generations from the draft model can be significantly reduced; when most retrieved code results in misses, the draft model can provide effective supplementation. Furthermore, in the prediction stage, we use the draft model as a proxy to avoid invoking the target model. In the expansion stage, we maximize the high-quality draft sequence to increase the length of accepted tokens, which ultimately reduces the number of verifications by the target model and improves overall decoding efficiency. Figure 7 illustrates an example where θ is set to 0.3. During the prediction phase, "`][n]`" is verified by the draft model and "`][n-`" is the snippet with confidence score greater than θ, both of which are retained as reusable retrieved code (yellow nodes). During the expansion phase, the leaf nodes of the retained tokens are first expanded based on the probability distribution obtained in the prediction phase (green nodes), and then the draft model furthermore expands the draft via generation (blue nodes) until the confidence score of all leaf tokens is lower than θ. By reusing high-quality

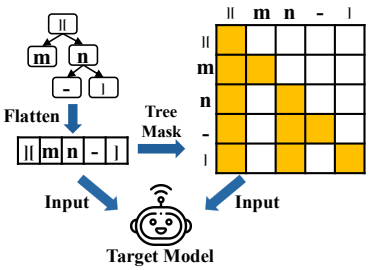


Figure 8: Illustration of the Tree Attention Mechanism.

parts of the retrieved code, the draft model requires only three generation steps in this example, one for prediction and two for expansion, and the target model can finally accept six tokens (i.e., "`][n]\n\telif`"). In contrast, using retrieval only results in three tokens verified by the target model (i.e., "`][n]`"), indicating that the retrieved draft does not reach the desired accepted length and in turn increases the number of target model verifications, while using generation only requires complete six inference steps from the draft model, resulting in substantially higher drafting cost.

3.5 Verification

After obtaining the draft code, to verify it in parallel, we employ tree attention [3, 30]. As illustrated in Figure 8, the tree-structured draft is first flattened into a sequence. Then, we construct an attention mask to ensure each token can only attend to its preceding nodes, thus guaranteeing the correctness of attention computation. Specifically, when constructing the attention mask, for each node, if a token is its ancestor, the corresponding value is set to `True` (as shown in yellow in each row of the matrix in the figure). Finally, the flattened sequence and the attention mask are fed into the target model for parallel computation to obtain the corresponding output of each token, and the verification is carried out via the process as illustrated in Section 2.1

4 EXPERIMENT

4.1 Research Questions

In this study, we address the following research questions:

RQ1: To what extent can `CacheCoder` accelerate code generation? This research question evaluates the efficiency of our approach and compares its overall performance with the baselines.

RQ2: What is the impact of each strategy in `CacheCoder` on performance? This question investigates the effectiveness of different strategies in our approach.

RQ3: How well does `CacheCoder` generalize? To address this question, we further evaluate the performance of our approach on additional datasets that were unseen during training.

4.2 Models and Datasets

We select two mainstream code LLMs **CodeLlama-34B** [32] and **Deepseek-Coder-33B** [8] as the backbone models. For the datasets, we used four subsets of **MBXP** [1], namely **MBCPP**, **MBGP**, **MBJSP**, and **MBPP** [2]. These subsets correspond to code generation tasks in C++, Go, JavaScript, and Python, respectively. Following prior work [9], we select 500 samples from each language version as the test set, and split the remaining data into training and validation sets at a 9:1 ratio. For the generalization study, we further evaluate `CacheCoder` on the **Multilingual HumanEval** dataset [1], including **HumanEval** [6], **HumanEval-Javascript**, and **HumanEval-Go**. The statistics of datasets are shown in Table 1.

Table 1: Dataset statistics.

Benchmark	Language	Train	Valid	Test
MBPP	Python	418	46	500
MBJSP	JavaScript	420	46	500
MBGP	Go	396	43	500
MBCPP	C++	314	34	500
HumanEval	Python	–	–	164
HumanEval-JS	JavaScript	–	–	161
HumanEval-Go	Go	–	–	160

4.3 Evaluation Metrics

Speedup. This metric evaluates the speedup ratio of the speculative decoding approach compared to autoregressive generation. Let the throughput (the number of tokens generated per second) of the target model with autoregressive decoding be TP_A, and the throughput of speculative decoding be TP_B. The speedup ratio of speculative decoding is given by TP_B/TP_A. A larger value indicates better acceleration performance.

Acceptance Length. This metric evaluates the average number of draft tokens accepted by the target model per decoding step in speculative decoding. A larger acceptance length indicates that the target model processes more tokens per step.

Acceptance Rate. This metric evaluates the average ratio between acceptance length and drafted length per step. A larger value indicates that the draft model has more effective inference steps.

Pass@k. This metric evaluates functional correctness by measuring the probability that at least one of the k generated samples passes all unit tests.

4.4 Baselines

We compare `CacheCoder` against the following baselines:

Autoregressive Decoding. Autoregressive decoding is the most basic decoding method. The target model decodes one token at each inference step based on the current context.

REST [10]. This approach retrieves relevant code snippets from the corpus based on the current context, and then the retrieved snippets are verified by the target model.

Self-speculative Decoding [43]. This approach leverages Bayesian optimization to select a subset of intermediate layers from the target model to serve as a draft model.

Eagle-3 [26]. This approach uses the hidden state of the target model as input to train a single-layer decoder from scratch. This single-layer decoder serves as the draft model.

4.5 Implementation Details

All experiments are run on four Nvidia 5090 GPUs. For construction of the draft model, α, β, S_t, C_t are set to 300, 500, 0.16, and

Table 2: The overall performance of CacheCoder and the baselines on the MBXP datasets across different sampling temperatures. The best performance is highlighted in bold, and the second-best is underlined.

Model	Approach	MBCPP				MBGP				MBJSP				MBPP				Average		
		Speedup	Acc. L	Acc. R	P@k	Speedup	Acc. L	Acc. R	P@k	Speedup	Acc. L	Acc. R	P@k	Speedup	Acc. L	Acc. R	P@k	Speedup	Acc. L	Acc. R
Temperature = 0, k=1																				
CodeLlama-34B	Autoregressive	1.00×	–	–	49.4	1.00×	–	–	41.0	1.00×	–	–	56.0	1.00×	–	–	54.2	1.00×	–	–
	REST	1.97×	1.36	–	49.4	1.89×	1.24	–	40.8	1.89×	1.31	–	56.0	1.79×	1.13	–	54.2	1.88×	1.26	–
	Self-speculative	1.27×	1.84	**82%**	49.6	1.37×	3.10	**84%**	40.8	1.38×	3.06	**84%**	56.4	1.31×	2.31	**82%**	54.4	1.33×	2.57	**83%**
	Eagle-3	2.18×	2.11	53%	49.6	2.18×	1.94	46%	41.2	2.25×	2.06	51%	56.2	2.00×	1.73	45%	54.2	2.15×	1.96	49%
	CacheCoder	**2.60×**	**5.38**	74%	49.6	**2.64×**	**5.05**	77%	40.8	**2.58×**	**5.07**	75%	56.2	**2.39×**	**4.43**	71%	54.2	**2.55×**	**4.98**	74%
Deepseek-Coder-34B	Autoregressive	1.00×	–	–	69.0	1.00×	–	–	53.0	1.00×	–	–	61.6	1.00×	–	–	61.8	1.00×	–	–
	REST	2.11×	1.51	–	69.0	1.97×	1.36	–	52.6	1.92×	1.30	–	61.6	1.86×	1.14	–	61.8	1.96×	1.32	–
	Self-speculative	1.58×	3.13	**81%**	69.0	1.45×	1.94	**82%**	52.8	1.54×	2.71	**82%**	61.8	1.50×	2.00	**80%**	61.8	1.51×	2.44	**81%**
	Eagle-3	2.18×	1.86	48%	69.0	2.04×	1.70	43%	52.6	2.13×	1.70	44%	61.8	1.99×	1.54	40%	61.8	2.08×	1.70	44%
	CacheCoder	**2.61×**	**6.04**	78%	69.0	**2.57×**	**5.38**	75%	52.8	**2.58×**	**5.47**	76%	61.6	**2.26×**	**4.40**	68%	62.0	**2.50×**	**5.32**	74%
Temperature = 0.8, k=5																				
CodeLlama-34B	Autoregressive	1.00×	–	–	67.6	1.00×	–	–	58.8	1.00×	–	–	67.8	1.00×	–	–	68.4	1.00×	–	–
	REST	1.89×	1.27	–	65.0	2.07×	1.49	–	57.2	1.83×	1.18	–	65.8	1.82×	1.12	–	67.6	1.92×	1.26	–
	Self-speculative	1.26×	1.84	**82%**	68.2	1.29×	1.99	**82%**	57.8	1.31×	2.29	**83%**	69.4	1.25×	1.62	**81%**	69.8	1.27×	1.93	**81%**
	Eagle-3	2.02×	1.87	48%	67.2	2.06×	1.91	46%	56.4	1.99×	1.76	44%	69.8	1.81×	1.58	41%	66.8	1.97×	1.78	45%
	CacheCoder	**2.33×**	**4.76**	67%	67.4	**2.40×**	**4.38**	69%	57.8	**2.32×**	**4.51**	67%	69.2	**2.15×**	**3.56**	62%	68.4	**2.30×**	**4.30**	66%
Deepseek-Coder-34B	Autoregressive	1.00×	–	–	80.4	1.00×	–	–	68.8	1.00×	–	–	77.8	1.00×	–	–	76.4	1.00×	–	–
	REST	2.13×	1.46	–	81.8	1.96×	1.32	–	68.2	1.85×	1.19	–	77.6	1.76×	1.06	–	77.6	1.92×	1.26	–
	Self-speculative	1.53×	2.46	**82%**	80.0	1.40×	1.61	**82%**	68.2	1.49×	2.09	**82%**	78.4	1.41×	1.57	**81%**	77.2	1.46×	1.93	**82%**
	Eagle-3	1.99×	1.66	43%	82.8	1.90×	1.49	39%	69.4	1.86×	1.49	38%	76.8	1.82×	1.38	36%	77.0	1.89×	1.50	39%
	CacheCoder	**2.50×**	**5.32**	70%	81.4	**2.35×**	**4.53**	69%	69.0	**2.32×**	**4.78**	68%	77.8	**2.08×**	**3.71**	61%	76.0	**2.31×**	**4.58**	67%

† Acc. L, Acc. R, and P@k denote Acceptance Length, Acceptance Rate, and Pass@k, respectively.

0.125, respectively, and then we perform 2000 rounds of Bayesian optimization to obtain the retained layers. Then, we distill the draft model for all programming languages, with a distillation learning rate of 5×10^{-5} and 20 epochs. During inference, we follow prior work [10] to set the maximum draft number as 64. The maximum draft depth d, the expansion number k, and the confidence score threshold θ are set to 10, 5, 0.3, respectively. We conducted a grid search over d from 6 to 14 with step 2, k from 4 to 8 with step 1, and θ from 0.1 to 0.6 with step 0.1, and the final values were chosen for best performance. For retrieval-based drafting, we use the first file from The Stack [20] as the retrieval corpus for each programming language, and this setting has also been adopted by previous work [45].

For Eagle-3, we train it on the same training data that we obtained. For REST, the retrieval corpus is same with CacheCoder. For other hyperparameters of the baseline, we follow the settings from the original paper.

Previous work [9] has shown that Code LLMs often generate excessive and useless code. To prevent such unnecessary generation, we employ a simple strategy based on string matching to terminate the generation early. For example, when generating Python code, we stop the generation if it produces keywords like "`def`" or "`if __name__ == '__main__' `".

5 Results

5.1 RQ1: Overall Performance

To answer this question, we systematically evaluate our approach on code generation tasks in different programming languages. We conducted experiments at different temperatures. When the temperature is 0 (i.e., greedy decoding), we generate one output for each prompt and report Pass@1. When the temperature is 0.8, we generate five outputs for each prompt and report Pass@5. Since retrieval-based approaches generate the draft in a single retrieval without an autoregressive generation step, we report only their acceptance length, excluding the acceptance rate.

Table 2 shows the overall performance of different approaches. From the results, it can be observed that CacheCoder outperforms all baselines across different programming languages, achieving the fastest speedup of 2.64× and 2.61× on CodeLlama-34B and Deepseek-Coder-33B, respectively. Compared with the second-fastest baseline Eagle-3 under greedy decoding, our approach improves the average speedup by 0.40× and 0.42× for CodeLlama-34B and Deepseek-Coder-33B, respectively. This result indicates that the reuse of well-trained intermediate layers and the KV cache of the target model allows CacheCoder to perform better with very limited distillation data and computational resources (i.e., 1.5k training data in this study v.s. over 500k data samples in the original implementation of Eagle-3). For CodeLlama-34B and Deepseek-Coder-33B, CacheCoder achieves an average acceptance length of 4.98 and 5.32 under greedy decoding across all datasets, significantly higher than Eagle-3 (1.96 and 1.70, respectively). Additionally, the average acceptance rate of CacheCoder is both 74% under greedy decoding, which also significantly outperforms Eagle-3 (49% and 45%, respectively). For Self-speculative Decoding, its overall speedup remains limited due to the high inference cost of the draft model. For example, for CodeLLama-34B, the draft model in self-speculative decoding retains 22 attention layers and 30 MLP layers, incurring

Table 3: Ablation study on the impact of retrieved drafts on CacheCoder under greedy decoding.

Model	Variant	MBCPP			MBGP			MBJSP			MBPP			Average		
		Speedup	Acc. L	Acc. R	Speedup	Acc. L	Acc. R	Speedup	Acc. L	Acc. R	Speedup	Acc. L	Acc. R	Speedup	Acc. L	Acc. R
CodeLlama-34B	CacheCoder	2.60×	5.38	74%	2.64×	5.05	77%	2.58×	5.07	75%	2.39×	4.43	71%	2.55×	4.98	74%
	w/o Retr. Code	2.36×	5.37	75%	2.40×	4.99	77%	2.47×	5.37	76%	2.22×	4.40	72%	2.36×	5.03	75%
	w/o BO	2.26×	3.99	59%	2.31×	3.60	60%	2.35×	3.96	62%	2.02×	2.98	55%	2.23×	3.63	59%
Deepseek-Coder-34B	CacheCoder	2.61×	6.04	78%	2.57×	5.38	75%	2.58×	5.47	76%	2.26×	4.40	68%	2.50×	5.32	74%
	w/o Retr. Code	2.39×	6.02	78%	2.34×	5.32	75%	2.36×	5.48	76%	2.13×	4.37	68%	2.30×	5.22	74%
	w/o BO	2.39×	4.64	63%	2.33×	4.00	61%	2.27×	3.93	61%	2.04×	3.12	55%	2.25×	3.93	60%

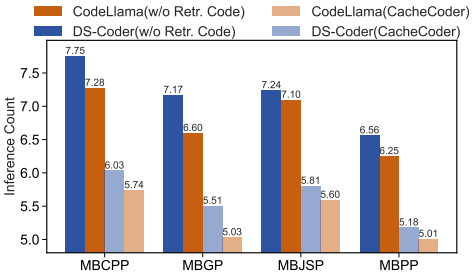


Figure 9: Impact of the retrieved code on the inference count of the draft model .

significant computational overhead during draft generation. In contrast, our draft model retains 5 attention layers and 6 MLP layers, which drastically reduces the draft generation cost and substantially boosts the overall speedup. For REST, the average acceptance length is very low, as most of the retrieved code fails to pass the target model validation. With the *Draft Expansion* introduced by CacheCoder, the acceptance length is substantially improved. For example, the acceptance length increases from 2.11 with REST to 5.38 with CacheCoder under greedy decoding on MBCPP.

Regarding the quality of the generated code, we observe that Pass@k scores of all approaches are nearly identical, with only slight differences, which may be due to precision issues during GPU computation. This phenomenon is consistent with the result of previous work [21], which has theoretically proven that speculative decoding is a lossless acceleration method.

Answer to RQ1: CacheCoder *outperforms recent state-of-the-art approaches across all datasets, and it can significantly accelerate code generation without sacrificing the generation quality.*

5.2 RQ2: Ablation Study

To investigate the effectiveness of different strategies in CacheCoder, we design two variants:

- **Without Retrieved Code (w/o Retr. Code)**: This variant does not reuse code snippets retrieved from the corpus and relies solely on the draft model for drafting.
- **Without Bayesian Optimization (w/o BO)**: This variant does not use Bayesian optimization to select the retained layers; instead, we select the first few layers of the target model for distillation, where the strategy was adopted in prior work [7, 28]. To keep the number of the selected layers as close as possible to that obtained by BO, we set the retained layers as follows: for CodeLlama-34B, where BO retains 5 attention layers and 6 MLP layers, we take the first 5 layers of the model as retained layers, which include 5 attention layers and 5 MLP layers; for Deepseek-Coder-33B, where BO retains 10 attention layers and 5 MLP layers, we select the first 7 layers of the model as retained layers, which include 7 attention layers and 7 MLP layers. The distillation data and training steps remain the same.

The performance of each variant under greedy decoding is shown in Table 3. From the results, we draw the following observations:

Impact of Retrieved Code. Comparing the second row of each model, we observe that when retrieved code is not used and only the draft model is used for drafting, the decoding speedup drops significantly by an average of 0.19× for CodeLlama-34B and 0.20× for Deepseek-Coder-33B. Although the acceptance length and acceptance rate remain almost unchanged, the absence of retrieved code forces the draft model to require more inference steps to generate drafts, thereby increasing the drafting time and reducing the speedup. Figure 9 compares the average number of inference steps per drafting step of the draft model with and without reuse of the retrieved code. Reusing retrieved code notably reduces the number of inference steps of the draft model, by an average of 1.46 steps for CodeLlama34B and 1.54 steps for Deepseek-Coder-33B. This indicates that combining reuse and generation achieves a good trade-off between efficiency and quality, improving drafting speed without sacrificing draft quality.

Impact of Layer Selection Strategy From the third row of each model, we see that using only the first few layers of the target model as retained layers leads to a decrease in speedup of 0.32× for CodeLlama34B and 0.25× for Deepseek-Coder-33B. This demonstrates that Bayesian optimization can select a set of layers that better preserve the capability of the target model, thus enabling more effective alignment under limited data.

Answer to RQ2: *All components in* CacheCoder *contribute positively to the performance. While BO can make the draft model more capable, reusing retrieved code can reduce the drafting cost.*

5.3 RQ3: Generalization Study

Table 4 shows the performance of different approaches on the untrained datasets (i.e., Multilingual HumanEval) under greedy decoding. The experimental results indicate that CacheCoder consistently remains the best-performing approach in all settings. For example, CacheCoder achieves an average speedup improvement

Table 4: The overall performance of CacheCoder on the Multilingual HumanEval datasets under greedy decoding.

Model	Approach	HumanEval-GO				HumanEval-JS				HumanEval				Average			
		Speedup	Acc. L	Acc. R	P@1	Speedup	Acc. L	Acc. R	P@1	Speedup	Acc. L	Acc. R	P@1	Speedup	Acc. L	Acc. R	
CodeLlama-34B	Autoregressive	1.00×	–	–	21.3	1.00×	–	–	44.1	1.00×	–	–	47.5	1.00×	–	–	
	REST	1.90×	1.25	–	21.3	1.90×	1.22	–	43.5	1.89×	1.20	–	47.5	1.89×	1.22	–	
	Self-speculative	1.38×	3.11	**83%**	21.3	1.36×	2.74	**84%**	44.1	1.30×	2.05	**82%**	47.5	1.34×	2.63	**83%**	
	Eagle-3	2.10×	2.11	45%	19.7	1.80×	1.67	42%	44.1	1.76×	1.47	39%	48.1	1.88×	1.75	42%	
	CacheCoder	**2.39×**	**4.53**	68%	21.3	**2.31×**	**4.30**	66%	44.1	**2.06×**	**3.47**	60%	47.5	**2.25×**	**4.10**	65%	
Deepseek-Coder-34B	Autoregressive	1.00×	–	–	29.3	1.00×	–	–	41.6	1.00×	–	–	54.2	1.00×	–	–	
	REST	1.96×	1.26	–	28.7	1.98×	1.30	–	42.2	2.00×	1.30	–	54.8	1.98×	1.28	–	
	Self-speculative	1.59×	3.09	**82%**	29.3	1.48×	2.08	**82%**	41.6	1.54×	2.49	**82%**	53.6	1.53×	2.55	**82%**	
	Eagle-3	1.93×	1.49	38%	29.3	1.96×	1.53	39%	41.6	1.77×	1.30	35%	53.6	1.88×	1.44	97%	
	CacheCoder	**2.43×**	**5.00**	71%	29.3	**2.45×**	**5.31**	74%	41.6	**2.15×**	**4.02**	64%	54.8	**2.34×**	**4.77**	69%	

```cpp
#include <bits/stdc++.h>
using namespace std;
/**
 * Write a c++ function to check whether the
sum of divisors are same or not.
 * > areequivalent(36, 57)
 * false
 * > areequivalent(2, 4)
 * false
 * > areequivalent(23, 47)
 * true
 */
bool areequivalent(int num1, int num2) {
    int sum1 = 0, sum2 = 0;
    for (int i=1; i<num1; i++) {
        if (num1%i == 0) sum1 += i;
    }
    for (int i=1; i<num2; i++) {
        if (num2%i == 0) sum2 += i;
    }
    return sum1 == sum2;
}
```

(a) REST

```cpp
#include <bits/stdc++.h>
using namespace std;
/**
 * Write a c++ function to check whether the
sum of divisors are same or not.
 * > areequivalent(36, 57)
 * false
 * > areequivalent(2, 4)
 * false
 * > areequivalent(23, 47)
 * true
 */
bool areequivalent(int num1, int num2) {
    int sum1 = 0, sum2 = 0;
    for (int i=1; i<num1; i++) {
        if (num1%i == 0) sum1 += i;
    }
    for (int i=1; i<num2; i++) {
        if (num2%i == 0) sum2 += i;
    }
    return sum1 == sum2;
}
```

(b) Self-draft model only

```cpp
#include <bits/stdc++.h>
using namespace std;
/**
 * Write a c++ function to check whether the
sum of divisors are same or not.
 * > areequivalent(36, 57)
 * false
 * > areequivalent(2, 4)
 * false
 * > areequivalent(23, 47)
 * true
 */
bool areequivalent(int num1, int num2) {
    int sum1 = 0, sum2 = 0;
    for (int i=1; i<num1; i++) {
        if (num1%i == 0) sum1 += i;
    }
    for (int i=1; i<num2; i++) {
        if (num2%i == 0) sum2 += i;
    }
    return sum1 == sum2;
}
```

(c) CacheCoder

Figure 10: Case study on the sources of accepted drafts across different approaches. Yellow represents drafts from the self-drafting model, green represents drafts from the retrieved code corpus.

of 0.41× compared to Eagle-3, and 0.36× compared to REST. In terms of acceptance length and acceptance rate, CacheCoder still significantly outperforms Eagle-3, with its acceptance rate exceeding 60% across all datasets. In contrast, Eagle-3 fails to achieve an acceptance rate of 40% in most cases. Specifically, when evaluated with Deepseek-Coder-34B as the target model under greedy decoding, Eagle-3 achieves an acceptance rate above 40% on its original benchmark, MBXP, but fails to do so on the unseen HumanEval datasets. For retrieval-based approach (i.e., REST), we observed a relatively small performance impact. However, due to the low quality of the retrieved code, the speedup of REST remains limited.

Answer to RQ3: *On unseen datasets,* CacheCoder *still achieves optimal performance compare to the-state-of-art approaches.*

6 Discussion

6.1 Case Study

To understand the effectiveness of CacheCoder, we conduct a case study. Figure 10 illustrates the accepted drafts under different approaches. For REST in Figure 10a, we find that only a small portion of the code snippets from the retrieved corpus are reused throughout the code generation process. In most cases, if all draft tokens are rejected, the final code is primarily generated by the target model. When relying solely on the draft model, illustrated in Figure 10b, we observe that at each decoding step, the draft model is able to provide high-quality drafts to the target model, meaning the acceptance length is greater than zero at every step. However, due to the autoregressive decoding of the draft model, its inference overhead grows with the count of inference step, thus accumulating significant drafting cost. In contrast, as shown in Figure 10c, CacheCoder significantly reduces the drafting time while maintaining high draft quality by reusing high-quality code from the retrieved corpus and using fewer inference steps to extend the retained code in each decoding step. The final results show that CacheCoder enables the draft model to draft with fewer inference steps, while the target model completes code generation with fewer verification steps.

Table 5: The ration of the accepted tokens' difference between the draft model and target model within ε.

Model	$\varepsilon = 0$	$\varepsilon = 1$
Deepseek-Coder-33B	85.12%	94.24%
CodeLlama-34B	86.61%	94.65%

6.2 Alignment Between Draft and Target Models

In CacheCoder, we view the tokens verified by the draft model as the hit part. However, this strategy needs a good alignment between the draft model and the target model. To assess the alignment

Table 6: Performance of CacheCoder combined with FastCoder under greedy decoding, where $\text{CacheCoder}_\text{F}$ denotes the combination of CacheCoder and FastCoder.

Model	Approach	MBCPP		MBGP		MBJSP		MBPP	
		Speedup	Acc. L	Speedup	Acc. L	Speedup	Acc. L	Speedup	Acc. L
CodeLlama-34B	REST	1.97×	1.36	1.89×	1.24	1.89×	1.31	1.79×	1.13
	FastCoder	1.99×	1.32	2.25×	1.55	1.91×	1.21	2.02×	1.27
	CacheCoder	2.60×	5.38	2.64×	5.05	2.58×	5.07	2.39×	4.43
	$\text{CacheCoder}_\text{F}$	**2.62**×	5.42	**2.89**×	5.33	**2.64**×	5.39	**2.46**×	4.52

degree, we conduct experiments on the validation dataset to compare the difference in accepted tokens between the draft and target models. Table 5 reports the results. For Deepseek-Coder-33B and CodeLlama-34B, the proportion of cases where the the draft model and the target model produce identical accepted tokens reaches 85.12% and 86.61%, respectively, and the proportion of cases where the difference in accepted token is at most one token exceeds 94% for both models. These results indicate a high degree of alignment between the draft and target models, and therefore, we can directly view the accepted tokens from the draft model as the hit part.

6.3 Combination with Other Retrieval Method

For code generation, more advanced retrieval-based approaches have been proposed, e.g., FastCoder [45], which maintains an additional cache on top of the retrieval corpus to store previously verified code segments and historical generation contexts, enabling more precise and faster retrieval. To investigate whether CacheCoder can be combined with FastCoder, we conduct additional experiments. Specifically, for the code segments from the cache of FastCoder, since they have either been previously verified or directly generated by the target model, we directly regard them as high-quality code, and use the draft model to continue them. Table 6 reports the performance of CacheCoder combined with FastCoder. The results show that while FastCoder consistently outperforms REST, integrating FastCoder with CacheCoder further improves performance. These findings show the effectiveness of combining CacheCoder with more efficient retrieval approaches.

6.4 Threats To Validity

Internal threats. A potential threat is the impact of the size of the retrieval corpus on CacheCoder. As the retrieval corpus increases in size, the probability of reusing retrieved code increases, but at the cost of longer retrieval times. In our experiments, we only used the code from the first file in The Stack [20] as the corpus. Future studies will further explore whether increasing the size of the retrieval corpus can further accelerate the inference speed of CacheCoder.

External threats. In this experiment, we focused on using a 30B parameter model, and did not include smaller models such as the 7B model. Nevertheless, larger models generally exhibit stronger capabilities and are more valuable in practical development. Furthermore, experiments conducted with different types of models all demonstrated the effectiveness of CacheCoder. Therefore, we believe that the results in this paper can provide a reasonable indication of the broad applicability of CacheCoder.

7 Related Work

Code generation has long been a task of significant interest to the research community [12, 17] and LLMs have achieved remarkable performance in this regard [16, 22, 23, 29, 31, 35, 38, 39, 41, 42]. However, these approaches primarily focus on improving generation quality rather than addressing the inference efficiency of LLMs for code generation. Guo et al. [9] observed that mainstream Code LLMs do not stop generation after producing complete code. To mitigate this issue, they trained a linear classifier to determine when to terminate generation. Nevertheless, Guo et al. does not fundamentally address the inefficiency caused by the autoregressive decoding process inherent to LLMs. To address the inefficiency of autoregressive decoding in LLMs, speculative decoding has been proposed [5, 21], which introduces a draft—verification paradigm. Based on this paradigm, a variety of approaches have been proposed to accelerate LLMs inference [3, 10, 25–27, 37, 43]. FastCoder [45] is a retrieval-based speculative decoding approach tailored for code generation. It proposes several strategies to skip unnecessary retrieval steps and employs a cache to store previously accepted tokens as well as tokens generated by the target model for retrieval.

In contrast to the above approaches, we are inspired by cache design in computer architecture and propose an effective strategy that integrates retrieval-based and generation-based drafts, achieving a balance between drafting speed and quality.

8 Conclusion

In this work, inspired by cache design in computer architecture, we propose CacheCoder. It treats the retrieved code, which can be fast to obtain but low in quality, as an L1 cache, and views the draft model, which is constructed from the target model and can generate higher-quality draft, as L2 cache. To fully leverage the efficiency of retrieval and the effectiveness of generation, we propose the *Preliminary Prediction* and *Draft Expansion* strategies, which first employs the draft model to predict high-quality parts of retrieved code and then performs expansion based on the predicted results. By reusing the retrieved code, the draft model can expand reused code with less inference steps, thereby achieving better balance between drafting quality and speed and finally reducing the number of the target model invocation. Experimental results show that CacheCoder exhibits excellent performance and consistently outperforms existing state-of-the-art approaches.

Data Availability

The code and data of this study are available at: https://doi.org/10.5281/zenodo.19338753.

References

[1] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868* (2022).

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *International Conference on Machine Learning*. PMLR, 5209–5235.

[4] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.

[5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318* (2023).

[6] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, et al. 2024. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12622–12642.

[8] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[9] Lianghong Guo, Yanlin Wang, Ensheng Shi, Wanjun Zhong, Hongyu Zhang, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. When to stop? towards efficient code generation in llms with excess token prevention. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1073–1085.

[10] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. 2024. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 conference of the North American chapter of the association for computational linguistics: Human language technologies (volume 1: long papers)*. 1582–1595.

[11] John L Hennessy and David A Patterson. 2011. *Computer architecture: a quantitative approach*. Elsevier.

[12] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[14] Yuxuan Hu, Ke Wang, Xiaokang Zhang, Fanjin Zhang, Cuiping Li, Hong Chen, and Jing Zhang. 2025. Sam decoding: Speculative decoding via suffix automaton. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12187–12204.

[15] Baizhou Huang, Shuai Lu, Xiaojun Wan, and Nan Duan. 2024. Enhancing large language models in coding through multi-perspective self-consistency. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1429–1450.

[16] Naman Jain, Tianjun Zhang, Wei-Lin Chiang, Joseph E Gonzalez, Koushik Sen, and Ion Stoica. 2023. Llm-assisted code cleaning for training accurate code generators. *arXiv preprint arXiv:2311.14904* (2023).

[17] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).

[18] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.

[19] Donald R Jones, Matthias Schonlau, and William J Welch. 1998. Efficient global optimization of expensive black-box functions. *Journal of Global optimization* 13, 4 (1998), 455–492.

[20] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. 2022. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533* (2022).

[21] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.

[22] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.

[23] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. Towards enhancing in-context learning for code generation. *arXiv preprint arXiv:2303.17780* (2023).

[24] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[25] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077* (2024).

[26] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840* (2025).

[27] Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. FastFixer: an efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.

[28] Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Duyu Tang, Kai Han, and Yunhe Wang. 2024. Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting. *Advances in Neural Information Processing Systems* 37 (2024), 11946–11965.

[29] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568* (2023).

[30] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 932–949.

[31] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. Clarifygpt: Empowering llm-based code generation with intention clarification. *arXiv preprint arXiv:2310.10996* (2023).

[32] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[33] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–22.

[34] Benjamin Spector and Chris Re. 2023. Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623* (2023).

[35] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2023. Test-case-driven programming understanding in large language models for better code generation. *arXiv preprint arXiv:2309.16120* (2023).

[36] Vinod Veeramachaneni. 2025. Large language models: A comprehensive survey on architectures, applications, and challenges. *Advanced Innovations in Computer Programming Languages* 7, 1 (2025), 20–39.

[37] Peiding Wang, Li Zhang, Fang Liu, Yinghao Zhu, Wang Xu, Lin Shi, Xiaoli Lian, Minxiao Li, Bo Shen, and An Fu. 2025. Reuse or Generate? Accelerating Code Editing via Edit-Oriented Speculative Decoding. *arXiv preprint arXiv:2506.02780* (2025).

[38] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F Bissyandé, and Xiaoguang Mao. 2023. Natural language to code: How far are we?. In *Proceedings of the 31st ACM joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 375–387.

[39] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F Bissyandé, and Xiaoguang Mao. 2024. Fusing code searchers. *IEEE Transactions on Software Engineering* 50, 7 (2024), 1852–1866.

[40] Shangwen Wang, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Yan Lei, and Xiaoguang Mao. 2023. Two birds with one stone: Boosting code generation and code search via a generative adversarial network. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (2023), 486–515.

[41] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro Von Werra, Arjun Guha, and Lingming Zhang. 2024. Selfcodealign: Self-alignment for code generation. *Advances in Neural Information Processing Systems* 37 (2024), 62787–62874.

[42] Prateek Yadav, Qing Sun, Hantian Ding, Xiaopeng Li, Dejiao Zhang, Ming Tan, Parminder Bhatia, Xiaofei Ma, Ramesh Nallapati, Murali Krishna Ramanathan, et al. 2023. Exploring continual learning for code generation models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 782–792.

[43] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024. Draft& verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the*

Association for Computational Linguistics (Volume 1: Long Papers). 11263–11282.

[44] Tianyi Zhang, Tao Yu, Tatsunori Hashimoto, Mike Lewis, Wen-tau Yih, Daniel Fried, and Sida Wang. 2023. Coder reviewer reranking for code generation. In *International Conference on Machine Learning*. PMLR, 41832–41846.

[45] Qianhui Zhao, Li Zhang, Fang Liu, Xiaoli Lian, Qiaoyuanhe Meng, Ziqian Jiao, Zetong Zhou, Jia Li, and Lin Shi. 2025. FastCoder: Accelerating Repository-level Code Generation via Efficient Retrieval and Verification. *arXiv preprint arXiv:2502.17139* (2025).

[46] Huanran Zheng and Xiaoling Wang. 2025. Faster speculative decoding via effective draft decoder with pruned candidate tree. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9856–9868.

[47] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2023. Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461* (2023).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009